Step 3 — Retries, Then a Safe Fallback, When a Model Call Fails — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — The retry is already there
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The retry is already there

The OpenAI client under `ChatOpenAI` **already retries** timeouts, connection errors, 429 and 5xx errors 2 times, with backoff (Doc04's error topic).

So don't add another retry layer — `.with_retry()` on top would turn one failure into up to 9 calls (Doc05: "one retry layer only").

What's missing is what happens **after** the retries run out. Today: a crash. You need a backup answer.

Which errors deserve a backup? Only the ones that pass on their own:

```
openai.APIConnectionError    # also covers APITimeoutError
openai.RateLimitError
openai.InternalServerError
```

Not `AuthenticationError` (a person must fix the key) and **not** a `KeyError` (a bug in your code). A backup for those would hide a real problem behind a polite reply.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
FALLBACK_REPLY = "Thanks ... we'll reply within one business day."

def canned_reply(inputs):
    log a warning
    return FALLBACK_REPLY

chain.with_fallbacks(
    [RunnableLambda(canned_reply)],
    exceptions_to_handle=TRANSIENT_ERRORS,
)
```

`RunnableLambda` turns the plain function into a step `.with_fallbacks()` accepts. The classifier gets its own backup: "general, not sure".

Put both wrappers in `chains/fallback.py`: `with_canned_reply(chain)` and `with_unsure_category(chain)`. Call them at the end of `build_reply_chain()` and `build_classifier_chain()`.

To know a fallback ran: the canned reply is always the same text, so `reply == FALLBACK_REPLY`. Store it in a new `TicketRecord` field, `used_fallback: bool = False` — the default keeps Step 1's code working.

Test all three cases in `tests/unit/test_fallback.py` — a real outage (a `ChatOpenAI` with `base_url="http://127.0.0.1:9"` and `max_retries=0`: nothing listens there), a working step that must **not** fall back, and a `KeyError` that must still raise.

Full solution: Show me the solution